Apuntes de clase — Proyecto 1: DNS Interceptor y capa física

- **Curso:** IC-7602 Redes — Semestre 2026-02
- **Estudiante:** Allan Bolaños Barrientos
- **Carné:** 2024145458

- **Fecha:** 2026-09-04

Proyecto 1 — reglas generales

- Vale **20%** de la nota, en grupos de **máximo 5 personas**.
- Cualquier indicio de copia (código o configuración tomada de internet sin reconocimiento al autor) se califica con **cero** y se procesa según el reglamento.
- La revisión es **presencial**, con evaluación escrita, y deben estar presentes todos los integrantes del grupo.
- Debe entregarse documentación clara para ejecutar el proyecto; se espera que **todo el grupo entienda** la implementación, no solo quien la escribió.
- Buenas prácticas esperadas: documentación interna/externa, estándares de código, diagramas de arquitectura y de flujo, pruebas unitarias.
- El lenguaje de implementación, si no lo especifica el profesor, lo elige cada grupo y debe quedar documentado. **No se permite Python.** Toda la documentación va en **Markdown**.
- Si la instalación no está automatizada → **cero**. Si el proyecto se entrega incompleto, la parte entregada debe estar completamente automatizada, si no → **cero**. Sin documentación → **cero**.
- Entrega por correo: copia del proyecto en `tar.gz` + link al repositorio, siguiendo el documento de lineamientos para envío de emails.
- Los grupos se autogestionan; el profesor no media si alguien no aporta.
- Para consultas (presenciales, correo, Telegram) hay que haber leído del tema y demostrado un intento de resolución propio antes de preguntar.
- El uso de IA generativa está regulado por la política del curso.

Objetivos del proyecto

- Implementar servicios de capa de aplicación sobre **UDP** y **TCP**.
- Servidores UDP y TCP en **C, Rust y Java** respectivamente.
- Procesar peticiones DNS según el formato de mensaje del **RFC 1035** (nota: en el audio se menciona "RFC 2929"/"RFC 1035" de forma confusa; el RFC que define el formato del header DNS —QR, OPCODE, etc.— es el **1035**).
- Transportar un servicio **no orientado a conexión (UDP)** sobre uno **orientado a conexión (TCP)**.
- Implementar un **API REST en Java**.
- Automatizar todo con Docker / Helm Charts / Terraform / Docker Compose / Kubernetes (o equivalentes).

- Implementar **health checks** para TCP y HTTP.

Contexto: cómo resuelve nombres una computadora

1. Toda máquina tiene un **resolver** (proceso/librería del SO que sabe resolver nombres) y un archivo tipo `resolv.conf` con la entrada `nameserver <IP>`.
2. El resolver primero revisa su **caché** local: - Si tiene el registro → responde al navegador, pero marcado como **no autoritativo** (no viene directo de la fuente). - Si no lo tiene → consulta al servidor DNS externo configurado (ej. `8.8.8.8`).
3. DNS corre sobre **UDP puerto 53**, es decir, sobre un protocolo de transporte **no confiable** (best-effort, sin garantía de entrega).
4. Se puede observar el tiempo de resolución DNS en el navegador, pestaña *Network* → *Timing* de una petición.

Arquitectura del proyecto: DNS Interceptor

Idea general: reemplazar el `nameserver` del resolver de la máquina cliente por un **DNS Interceptor** propio, de forma transparente (el navegador y el resolver no deben notar el cambio). Es el mismo principio detrás de servicios como **Route 53** de AWS, aplicado a pequeña escala.

Componentes:

- **Cliente DNS**: la máquina normal, sin cambios salvo apuntar su `nameserver` al interceptor.
- **DNS Interceptor**: contenedor Docker con una aplicación en **Rust** escuchando en **UDP/53**. Recibe los paquetes DNS y decide qué hacer con ellos.
- **DNS API**: componente REST (Java) al que el interceptor reenvía las consultas que no puede resolver localmente.

Nota práctica: el puerto UDP/53 suele estar protegido por antivirus, firewalls del SO y software de seguridad — hay que investigar cómo levantarlo igual (parte del ejercicio de investigación).

Formato del mensaje DNS (RFC 1035, sección de header)

El header del mensaje DNS es binario (usa bytes sin representación gráfica ASCII), de longitud fija, y entre otros campos incluye:

- **ID**: 2 bytes, identificador de la transacción.
- Byte siguiente incluye, de más significativo a menos significativo:
- **QR** (1 bit): `0` = query, `1` = respuesta.
- **OPCODE** (4 bits): tipo de consulta. `0` = *standard query* (existen otros valores como *inverse query*, etc.).
- Resto de banderas (AA, TC, RD, ...).

Regla de procesamiento que pide el proyecto:

- Se extrae el **tercer byte** del header del paquete recibido.
- Se examina el **bit más significativo** → ese es **QR**.

- Si `QR = 0` → es una consulta (query), se sigue examinando.
- Si `QR ≠ 0` → no se procesa nada especial con ese paquete.
- Se examinan los siguientes 4 bits → **OPCODE**.
- Si `QR = 0` **y** `OPCODE = 0` → es un **query estándar**: se debe examinar en detalle e identificar el host que se está resolviendo (esta parte queda pendiente de profundizar en la siguiente clase).
- Si el paquete **no** es un query estándar (`QR` u `OPCODE` distinto de lo esperado):
  1. Codificar la solicitud completa en **Base64**.
  2. Enviarla por **HTTP(S) POST** al endpoint del componente **DNS API**.
  3. La respuesta llega también codificada en Base64.
  4. Decodificarla y devolverla al cliente que hizo la consulta original.

Por qué Base64 aquí (idea clave del curso)

- DNS es un protocolo **binario**: usa bytes sin representación gráfica en ASCII.
- HTTP es un protocolo de **texto plano**: solo transporta caracteres imprimibles.
- Para meter un mensaje binario dentro de un protocolo de texto, se **codifica en Base64** (convierte bytes arbitrarios a un alfabeto de caracteres imprimibles). Prácticamente todo lenguaje trae Base64 en su librería estándar.
- Consecuencia interesante del diseño: se está transportando
- un protocolo **no confiable (UDP/DNS)** sobre uno confiable (**TCP** dentro de HTTP), y
- un protocolo con **caracteres no imprimibles** sobre uno que solo acepta **caracteres imprimibles** (HTTP/Base64).
- Todo esto debe pasar desapercibido para el cliente final: ni el resolver ni el navegador deben notar que hay un interceptor en medio. Por eso es crítico leer bien el RFC y no equivocarse de bit al armar/parsear el header.

Pendiente para la próxima clase

- Qué hacer exactamente al identificar el host en un query estándar (`QR=0`, `OPCODE=0`) — quedó a medias en la grabación.
- Profundizar en capa física.